Noto Peninsula: An Interactive 3D Landscape

Project Proposal

Group 12
LEE Hyunjin, TAN Jiming

HKUST - COMP5411 - Fall 2026

Background

LEE Hyunjin has no prior experience in computer graphics, but has some experience in JavaScript.

TAN Jiming has a background in civil engineering and no prior graphics programming experience.

Project description

Noto Peninsula lies in Ishikawa Prefecture in central Japan and extends into the Sea of Japan (Figure 1). It is a popular tourist destination, known for its scenic coastline and mountainous landscape. Our project, *Noto Peninsula: An Interactive 3D Landscape*, aims to help people explore the region virtually and better understand its terrain. By allowing users to view the landscape from different angles and scales, the model will help them see how its mountains, valleys, and coastline relate to each other.

Our selected modeling region covers approximately 33.23×23.14 km, with the model base extending to 100 m below sea level (Figure 2). We use elevation data with a spatial resolution of 1 m, downloaded from the Geospatial Information Authority of Japan (GSI), to represent the terrain. Terrain materials are assigned from the Geological Survey of Japan's geological map of the Suzu-Misaki, Noto-Iida and Horyuzan area, supported by borehole logs from NIED K-NET/KiK-net and the MLIT Noto Reconstruction Office. These are grouped into a small set of rendering categories rather than exact engineering properties. We will develop the application using Three.js and WebGL, with custom GLSL shaders for the planned visual effects. Users will be able to rotate, pan, and zoom the camera to inspect the region at different scales. Rock and soil textures will distinguish terrain materials, with blending at appropriate boundaries to create natural transitions. Illustrative trees and grass will add detail to the land surface. An animated sea will show waves, highlights, and foam washing up along the shore. Users can set the date and time and choose from several weather presets to see how the landscape's appearance changes.

The expected outcome is **an interactive browser application** that presents the Noto Peninsula landscape. Adaptive terrain detail will aim to maintain smooth navigation at different viewing scales and on different computers. Development and testing are planned over nine weeks, from course Week 4 to Week 10. The final deliverables will include the working application, a demonstration video, and a project report describing the implementation and evaluating visual quality and rendering performance.



Figure 1: Location of the Noto Peninsula.

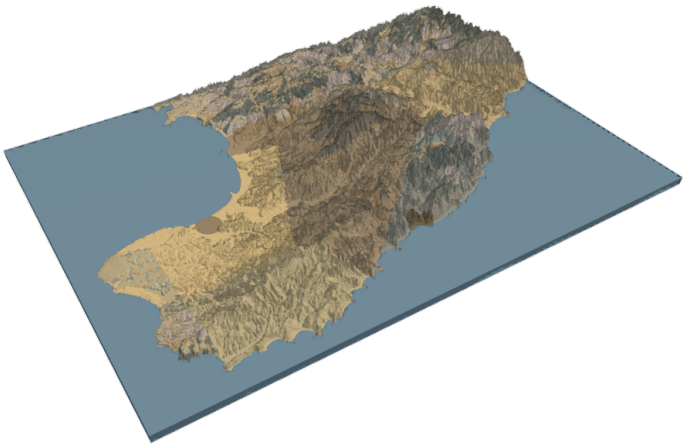


Figure 2: Terrain geometry.

Existing technology

There are several tools online that let people view real terrain in 3D. Google Earth stream global elevation data and draped satellite imagery in the browser, and the Geospatial Information Authority of Japan's GSI Maps 3D can display and export terrain for any area of Japan. These tools prioritize geographic accuracy and wide coverage, so terrain is usually shown with photographs or simple shading. Game engines such as Unreal Engine and Unity provide rich terrain materials, water, and weather effects, but they typically use artist-made or procedural terrain and require large installed applications.

Our project builds on established real-time rendering techniques: texture splatting for blending terrain materials, instanced rendering for vegetation, Gerstner waves for ocean surfaces, approximate solar-position models for daylight, shadow mapping, and tile-based level of detail in the style of chunked LOD and CDLOD. Rather than proposing a new algorithm, we aim to combine these techniques with real elevation and geological data for a single region. The result will be a lightweight browser application that is both geographically grounded and visually expressive, showing how the Noto Peninsula's terrain, coastline, and lighting change over time.

Key challenges

- **Terrain materials and transitions.** Different rock and soil materials should remain visually distinct while forming natural transitions. Steep slopes and changes in viewing scale can reveal texture stretching, repetition, or abrupt material boundaries. We will use texture mapping and blend neighboring textures in a fragment shader, with suitable texture scales and blending widths to reduce visible seams.
- **Efficient animated vegetation.** Many trees and grass patches can slow the browser, while identical repeated shapes can look artificial. We will place illustrative vegetation using instanced meshes, reduce its density and geometric detail with viewing distance, and add subtle wind motion to nearby foliage.
- **Animated coastal water.** The sea surface should show moving waves and highlights while remaining consistent with the coastline. Near shore, foam should advance and fade without visibly cutting through the terrain. We will implement Gerstner waves with matching surface normals, then animate a narrow foam band using a precomputed shoreline-distance map and reduce wave amplitude at the coast.
- **Date, weather and dynamic shadows.** Solar angle changes with date and time, while weather changes scene brightness; these controls must keep lighting and shadows coherent. Low-angle sunlight and close-up views can also expose shadow artifacts across this large region. We will calculate an approximate sun position from the selected date and time, adjust lighting for clear, cloudy, and rainy presets, and tune shadow-map coverage and resolution.

- **Adaptive terrain detail.** Rendering the full region at the finest mesh resolution would impose a substantial memory and processing cost. A coarse mesh is faster but may remove important terrain features, while sudden resolution changes can cause visible popping or gaps. We will select from several mesh resolutions using viewing distance and frame rate, while maintaining matching boundaries and limiting frequent LOD changes.

Timeline

The schedule below follows the course-week numbering used in our proposal presentation. Terrain modeling is already complete.

Week	**Milestone**	**Details / Tasks**
Completed	**Proposal** / Terrain modeling	Process raw elevation data, generate the terrain mesh, and complete the model geometry and camera controls.
4	Materials	Apply ground and water textures and blend neighboring terrain materials.
5	**Progress** / Vegetation	Add illustrative trees and grass using instancing, distance-based detail, and gentle animation.
6	Animated water	Implement Gerstner waves, surface-normal lighting, and a shoreline foam band to suggest waves washing up and receding.
7–8	**Demo** / Sunlight and weather	Add date, time, and weather controls; compute the sun angle and update lighting, shadows, and simple rain effects.
9	Adaptive detail	Select mesh resolution according to viewing distance and frame rate. Check continuity between levels of detail.
10	**Final** / Integration and testing	Check material seams, coastlines, shadows, and LOD transitions. Evaluate performance and prepare a stable demo, presentation video, and project report.

Design and implementation details

Materials and vegetation. We will map the available rock and soil categories to suitable textures and blend neighboring materials in a fragment shader. Illustrative trees and grass will be placed on land using instanced meshes, with their density and mesh detail reduced at longer viewing distances. Nearby foliage will sway gently through vertex animation.

Animated water. We will animate a separate sea mesh at sea level with a few Gerstner waves, updating its surface normals for moving highlights. A precomputed shoreline-distance map will drive a narrow white foam band that advances onto the shore and fades as it recedes. We will lower wave amplitude near land to avoid obvious intersections. This is a visual wash-up effect rather than a physical breaking-wave simulation.

Sunlight and weather. Users will choose a calendar date and time; an approximate sun-position calculation for the Noto Peninsula will set solar azimuth and elevation, so seasonal as well as daily changes affect lighting. Clear, cloudy, and rainy presets will adjust sky color and light intensity, with lightweight rain particles for the rainy setting. We will use Three.js shadow mapping and tune its coverage and resolution for regional and close-up views, particularly at low sun angles.

Adaptive detail. We will preload a 16 m terrain layer for the full modeling area before camera navigation begins, so moving the viewpoint does not reveal empty tiles. For visible tiles, we will use their projected size on screen and the recent frame rate to decide where finer terrain data should be loaded asynchronously. The 16 m layer will remain visible until a higher-resolution tile is ready to replace it.

Integration and testing. We will combine these features in one navigable application and test both full-region and close-up views. We will inspect material boundaries, water–terrain contact, shadows, and tile transitions, while recording frame rate and loading behavior at representative camera positions.

AI assistance

AI has helped us process the raw elevation data into terrain vertex files. We use AI to discuss possible approaches to the technical challenges and to generate parts of the implementation code from our instructions. The group will choose the methods, review and adapt AI-assisted code, and test the integrated application, including its geometry, visual effects, and performance. We will identify the AI-assisted components in the final report.